

Generic Hardware Access in Linux

This article, which is part of the series on Linux device drivers, talks about accessing hardware in Linux.

Part—7

Shweta was all jubilant about her character driver achievements, as she entered the Linux device drivers laboratory on the second floor of her college. Many of her classmates had already read her blog and commented on her expertise. And today was a chance to show off at another level. Till now, it was all software—but today's lab was on accessing hardware in Linux.

In the lab, students are expected to learn 'by experiment' how to access different kinds of hardware in Linux, on various architectures, over multiple lab sessions. Members of the lab staff are usually reluctant to let students work on the hardware straight away without any experience—so they had prepared some presentations for the students, which you can get from http://profession.sarika-pugs.com/index.php?pagefile=linux_drivers.

Generic hardware interfacing

As every one settled down in the laboratory, lab expert Priti started with an introduction to hardware interfacing in Linux. Skipping the theoretical details, the first interesting slide was about generic architecture-transparent hardware interfacing (see Figure 1). The basic assumption is that the architecture is 32-bit. For others, the memory map would change accordingly. For a 32-bit address bus, the address/memory map ranges from 0 (0x00000000) to '2³² - 1' (0xFFFFFFFF). An architecture-independent layout of this memory map would be like what's shown in Figure 1—memory (RAM) and device regions (registers and memories of devices) mapped in an interleaved fashion. These addresses

actually are architecture-dependent. For example, in an x86 architecture, the initial 3 GB (0x00000000 to 0xBFFFFFFF) is typically for RAM, and the later 1 GB (0xC0000000 to 0xFFFFFFFF) for device maps. However, if the RAM is less, say 2GB, device maps could start from 2GB (0x80000000).

Run '`cat /proc/iomem`' to list the memory map on your system. Run '`cat /proc/meminfo`' to get the approximate RAM size on your system. Refer to Figure 2 for a snapshot. Irrespective of the actual values, the addresses referring to RAM are termed as physical addresses, and those referring to device maps as bus addresses, since these devices are always mapped through some architecture-specific bus—for example, the PCI bus in the x86 architecture, the AMBA bus in ARM architectures, the SuperHyway bus in SuperH architectures, etc.

All the architecture-dependent values of these physical and bus addresses are either dynamically configurable, or are to be obtained from the data-sheets (i.e., hardware manuals) of the corresponding architecture processors/controllers. The interesting part is that in Linux, none of these are directly accessible, but are to be mapped to virtual addresses and then accessed through them—thus making the RAM and device accesses generic enough. The corresponding APIs (prototyped in `<asm/io.h>`) for mapping and unmapping the device bus addresses to virtual addresses are:

```
void *ioremap(unsigned long device_bus_address, unsigned long
device_region_size);
void iounmap(void *virt_addr);
```